

Механизмы разграничения прав доступа в ОС Linux. Ч2

М. Е. Исмагулов

Югорский государственный университет

20 сентября 2026 г.

- 1 Механизм проверки прав доступа у субъекта к объекту.
- 2 Владельцы папок и файлов, права доступа к файлам и папкам, маска создания файлов и папок.

Механизм проверки прав доступа у субъекта
к объекту.

Субъект, объект и операция доступа

Ключевая идея

Права доступа проверяются не абстрактно «у пользователя», а в момент попытки **субъекта** выполнить конкретную **операцию** над конкретным **объектом**.

СУБЪЕКТ	ОПЕРАЦИЯ	ОБЪЕКТ
процесс	read	файл
процесс	write	каталог
процесс	execute	сокет
процесс	delete	процесс

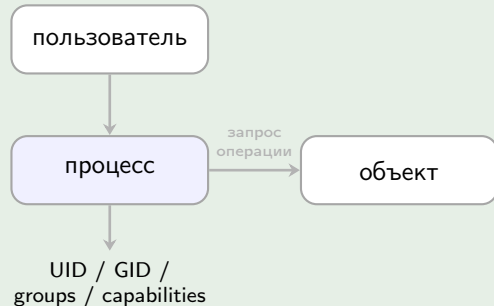
Непосредственным субъектом доступа в Linux является не пользователь, а **выполняющийся процесс**, действующий с определёнными учётными данными (UID, GID, дополнительные группы, capabilities).

Наивная модель против реальной

Наивная модель

пользователь → файл

Реальная модель



Эта модель в дальнейшем объясняет работу `sudo`, механизма `setuid` и `capabilities`: процесс наследует учётные данные, но эти данные могут быть изменены в момент запуска или во время выполнения.

Какие данные участвуют в проверке

Формализация

$$\text{Access}(S, O, A) \rightarrow \{\text{Allow}, \text{Deny}\}$$

где S — subject (субъект), O — object (объект), A — requested action (запрошенная операция)

Субъект — процесс

- filesystem UID
- filesystem GID
- дополнительные группы
- capabilities
- контекст безопасности

Объект — например, inode файла

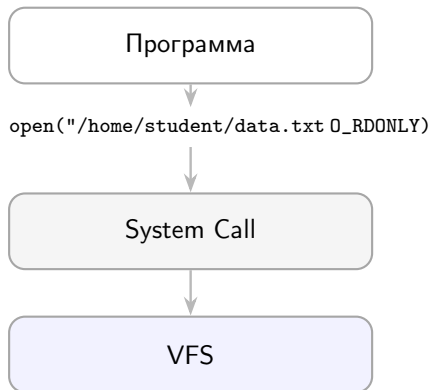
- UID владельца
- GID группы
- mode bits
- ACL
- атрибуты безопасности

requested operation: read / write / execute

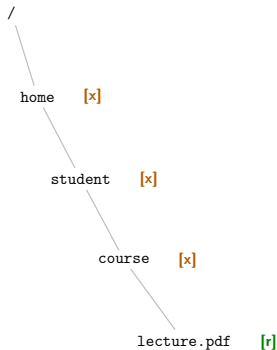
Где происходит проверка доступа

Ключевой тезис

Прикладная программа **не решает**, имеет ли процесс доступ к файлу. Решение принимается **ядром** операционной системы.



Чтобы открыть файл, нужно сначала дойти до него



Внимание

Наличие права чтения на файл **недостаточно**, если процесс не имеет права поиска (x) хотя бы для одного каталога в пути.

Практический пример

```
$ cat /srv/course/lecture.txt  
cat: /srv/course/lecture.txt: Permission denied
```

Хотя права на сам файл выглядят вполне разрешающими:

```
-rw-r--r-- lecture.txt
```

Причина отказа может обнаружиться на уровень выше:

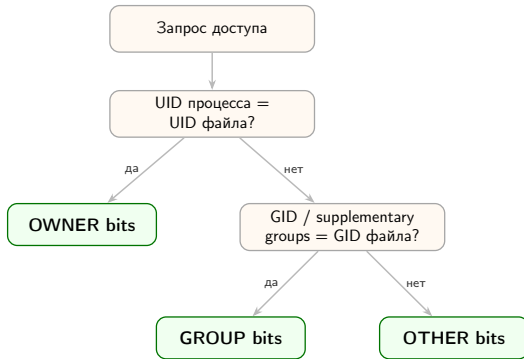
```
drwx----- course/
```

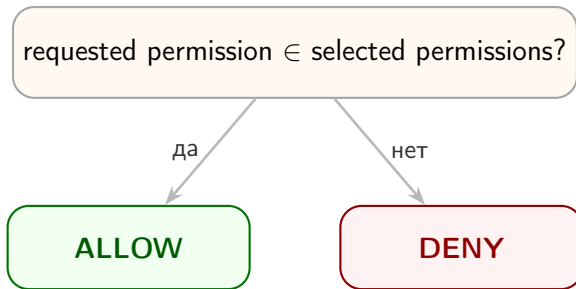
Разбор ситуации

Каталог `course` не даёт права `x` никому, кроме владельца (`rwX---`). Даже при полностью открытых правах на `lecture.txt` процесс не может *дойти* до файла: попытка поиска внутри `course` отклоняется ещё на этапе разрешения пути, до того как права самого файла вообще будут рассмотрены.

Базовый алгоритм DAC: owner → group → other

DAC (Discretionary Access Control) — дискреционная модель управления доступом, в которой права задаются владельцем объекта и проверяются на основании принадлежности субъекта к одному из трёх классов: владелец, группа-владелец, прочие.





Важно понимать

Это **не** последовательность «если owner не разрешил — попробуем group». Сначала выбирается **ровно один** класс субъекта (owner, group или other), и только затем проверяются права **только этого класса**. Если выбранный класс не даёт нужного права — результат DENY, даже если другой класс, к которому процесс не относится, разрешил бы операцию.

student принадлежит группе developers

```
-r--rw---- report.txt
```

```
owner: student  group: developers
```

Может ли student записывать report.txt?

student принадлежит группе developers

```
-r--rw---- report.txt
```

```
owner: student  group: developers
```

Может ли student записывать report.txt?

НЕТ

Права разных классов не складываются

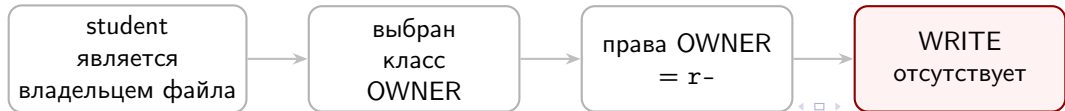
student принадлежит группе developers

```
-r--rw---- report.txt
```

```
owner: student  group: developers
```

Может ли student записывать report.txt?

НЕТ



POSIX ACL: когда owner/group/other недостаточно

Классическая модель

owner	rw-
group	r-
other	--

ACL

owner	rw-
user:anna	r-
user:ivan	rw-
group	r-
group:dev	rw-
mask	rw-
other	--

Ключевая мысль

ACL позволяет назначать права **конкретным** пользователям и группам, не изменяя владельца объекта.

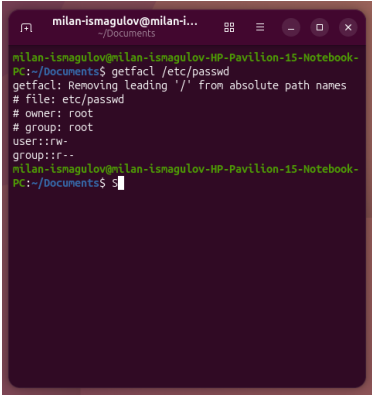
ACL на практике: вывод getfacl

```
$ getfacl report.txt
```

```
user::rw-  
user:anna:r--  
group::r--  
mask::r--  
other::---
```

ACL-проверка учитывает владельца, именованные пользовательские записи, группы и ACL_MASK — запись, ограничивающую эффективные права большинства ACL-записей (кроме owner и other).

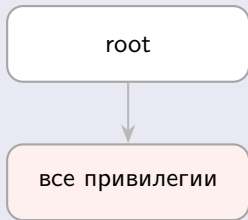
Проверьте у себя: `getfacl /etc/passwd` или создайте тестовый файл и примените `getfacl` к нему.



```
milan-ismagulov@milan-i...  
~/Documents  
milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-  
PC:~/Documents$ getfacl /etc/passwd  
getfacl: Removing leading '/' from absolute path names  
# file: etc/passwd  
# owner: root  
# group: root  
user::rw-  
group::r--  
milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-  
PC:~/Documents$
```

Почему root может больше: Linux capabilities

Традиционный UNIX



Linux

```
CAP_CHOWN  
CAP_DAC_OVERRIDE  
CAP_DAC_READ_SEARCH  
CAP_NET_ADMIN  
...  
CAP_...
```

Начиная с Linux 2.2, традиционные root-привилегии разделены на отдельные независимо включаемые и отключаемые capabilities.

Capabilities, важные для файловых проверок

CAP_DAC_OVERRIDE

Позволяет процессу обходить стандартные проверки прав доступа DAC на чтение, запись и выполнение файлов.

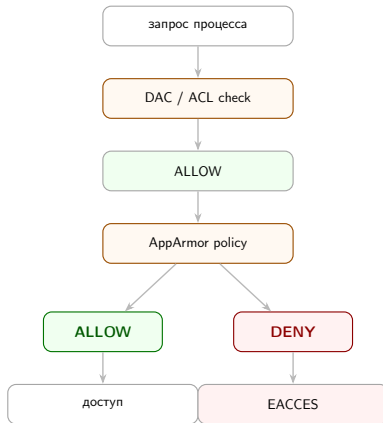
CAP_DAC_READ_SEARCH

Позволяет процессу обходить проверки права чтения файлов, а также проверки права чтения и поиска (search/execute) по каталогам.

Ключевой вывод

Проверка прав зависит от **полномочий процесса**, а не только от имени пользователя.

Дополнительный уровень контроля: AppArmor

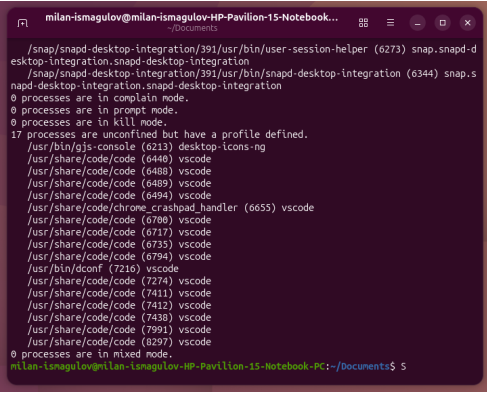


AppArmor: ключевая мысль

Важно

Успешная проверка DAC ещё не гарантирует доступ: операция может быть запрещена политикой Mandatory Access Control (MAC), действующей поверх стандартной UNIX-модели прав.

В Ubuntu AppArmor является реализацией фреймворка Linux Security Module (LSM) и дополняет традиционную UNIX DAC-модель. Он применяет профили, ограничивающие поведение конкретных программ, и может запретить операцию, которую обычные файловые права разрешают.

A terminal window titled 'milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-...' showing the output of the 'snap status' command. The output lists the status of various snap processes, including 'snapd-desktop-integration' and 'vscodesnapd'. It shows that 0 processes are in complain or prompt mode, 0 are in kill mode, and 17 are unconfined but have a profile defined. The profiles listed include 'desktop-icons-ng', 'vscodesnapd', and 'chrome_crashpad_handler'. The terminal ends with the prompt 'milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-PC:~/Documents\$' and a cursor.

```
milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-...  
~/Documents  
/snap/snapd-desktop-integration/391/usr/bin/user-session-helper (6273) snap.snapd-d  
esktop-integration.snapd-desktop-integration  
/snap/snapd-desktop-integration/391/usr/bin/snapd-desktop-integration (6344) snap.s  
napd-desktop-integration.snapd-desktop-integration  
0 processes are in complain mode.  
0 processes are in prompt mode.  
0 processes are in kill mode.  
17 processes are unconfined but have a profile defined.  
/usr/bin/gjs-console (6213) desktop-icons-ng  
/usr/share/code/code (6446) vscodesnapd  
/usr/share/code/code (6488) vscodesnapd  
/usr/share/code/code (6489) vscodesnapd  
/usr/share/code/code (6494) vscodesnapd  
/usr/share/code/chrome_crashpad_handler (6655) vscodesnapd  
/usr/share/code/code (6706) vscodesnapd  
/usr/share/code/code (6717) vscodesnapd  
/usr/share/code/code (6735) vscodesnapd  
/usr/share/code/code (6794) vscodesnapd  
/usr/bin/dconf (7216) vscodesnapd  
/usr/share/code/code (7274) vscodesnapd  
/usr/share/code/code (7411) vscodesnapd  
/usr/share/code/code (7412) vscodesnapd  
/usr/share/code/code (7438) vscodesnapd  
/usr/share/code/code (7991) vscodesnapd  
/usr/share/code/code (8297) vscodesnapd  
0 processes are in mixed mode.  
milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-PC:~/Documents$
```

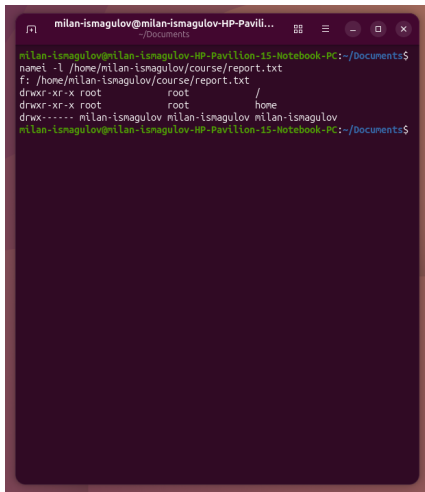
Как ядро отвечает на вопрос «можно ли открыть файл?»

```
$ cat /srv/course/report.txt  
cat: /srv/course/report.txt:  
Permission denied
```

- 1 Можно ли пройти по пути?
- 2 Какие credentials у процесса?
- 3 Какой класс DAC применяется?
- 4 Разрешена ли операция?
- 5 Есть ли ACL?
- 6 Есть ли capability?
- 7 Разрешает ли AppArmor?

**Permission denied — результат механизма проверки,
а не объяснение причины отказа.**

Практические инструменты диагностики

A terminal window with a dark background and light green text. The window title is 'milan-ismagulov@milan-ismagulov-HP-Pavili...' and the current directory is '~/Documents'. The user has entered the command 'namei -l /home/milan-ismagulov/course/report.txt'. The output shows the file path and permissions for each component of the path: 'f: /home/milan-ismagulov/course/report.txt', 'drwxr-xr-x root root /', 'drwxr-xr-x root root home', and 'drwx----- milan-ismagulov milan-ismagulov milan-ismagulov'.

```
milan-ismagulov@milan-ismagulov-HP-Pavili...  
~/Documents  
milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-PC:~/Documents$  
namei -l /home/milan-ismagulov/course/report.txt  
f: /home/milan-ismagulov/course/report.txt  
drwxr-xr-x root      root      /  
drwxr-xr-x root      root      home  
drwx----- milan-ismagulov milan-ismagulov milan-ismagulov  
milan-ismagulov@milan-ismagulov-HP-Pavilion-15-Notebook-PC:~/Documents$
```

```
id  
namei -l /srv/course/report.txt  
getfacl /srv/course/report.txt  
aa-status  
journalctl
```

Особенно полезен `namei -l` — он визуально показывает права всех компонентов пути, что позволяет сразу увидеть, на каком именно уровне произошёл отказ.

Владельцы папок и файлов, права доступа к файлам и папкам, маска создания файлов и папок.

